

Отчёт по аудиту бэкенда **Amazon_Clonе**

Дата: 2026-08-21 **Область:** Django REST API (backend, модели, сервисы, конфиг)

Резюме

Архитектура проекта сильная и осознанная:

- **Сервисный слой** (`View -> serializer -> service -> ORM`) — бизнес-логика вынесена из `views`.
- **Конечный автомат** статусов заказов с валидацией переходов.
- **Snapshot-цены и адреса** в `OrderItem` / `Order` (заказ = immutable-документ).
- **Денормализация:** `min_price` / `max_price` / `rating` / `reviews_count` / `views_count`.
- **Конкурентная безопасность:** `select_for_update` + `transaction.atomic`.
- **PostgreSQL FTS:** `SearchVector` + `GinIndex`, сигнал обновляет поисковый индекс.
- **Тесты:** кастомный runner, фабрики, тесты по слоям (`api/models/services/queriesets/signals`).

Найденные проблемы касаются в основном **безопасности, конкурентной корректности** и **неиспользуемых оптимизаций**.

Критично

1. `select_for_update().get_or_create()` — скрытый рантайм-баг

Файл: `apps/inventory/services/inventory_service.py` (`reserve_stock`, `get_or_create_stock`)

```
Stock.objects.select_for_update().get_or_create(variant=..., defaults=...)
```

Django **не поддерживает** комбинацию `select_for_update()` + `get_or_create()`: при создании новой строки выбрасывается `NotSupportedError`. Код «выглядит правильным», но падает на ветке «сток ещё не создан».

Фикс: сначала `get_or_create()` без блокировки, затем перечитать строку с `select_for_update()` (как уже сделано в `adjust_stock`).

2. Webhook оплаты полностью открыт

Файл: `apps/payments/api_views/payment_views.py` (`PaymentWebhookView`)

- `AllowAny` + `authentication_classes = []`.
- Нет проверки подписи, белого списка IP, `throttle` на эндпоинт.
- Любой `POST /api/v1/payments/webhook/` с `external_id` + `status="succeeded"` подтверждает заказ **без реальной оплаты**.

Фикс:

- HMAC-подпись (секрет из `.env`) + заголовок `Idempotence-Key`.
 - Проверка `X-Forwarded-For` / белый список IP провайдера.
 - Лёгкий `AnonRateThrottle` на сам эндпоинт.
 - В mock-режиме — явный флаг «только для теста».
-

3. JWT ротация без черного списка

Файл: `config/settings.py` (`SIMPLE_JWT`)

- `ROTATE_REFRESH_TOKENS=True` + `BLACKLIST_AFTER_ROTATION=True`.
- Но в `INSTALLED_APPS` **нет** `rest_framework_simplejwt.token_blacklist`.
- Старые refresh-токены не аннулируются → утечка refresh-токена не гасится.

Фикс: добавить `"rest_framework_simplejwt.token_blacklist"` в `INSTALLED_APPS` и прогнать миграции.

4. Генерация `order_number` через `MAX()+1` — race condition

Файлы:

- `apps/orders/models/order.py` (`save()`)
- `apps/orders/services/order_service.py` (`create_from_cart`)

```
max_seq = Order.objects.aggregate(max_seq=models.Max('_order_number_seq'))  
['max_seq'] or 0
```

Два параллельных заказа могут прочитать одинаковый `MAX` → дубликаты `ORD-000001`.

Фикс:

- PostgreSQL `Sequence` (отдельный счётчик) либо UUID/CUID-префикс.
 - Генерацию номера из `save()` вынести в сервис, выполняемый внутри одной транзакции.
-

5. Секреты и артефакты в репозитории

- `config/settings.py` — захардкоженный `SECRET_KEY` в качестве дефолта.
- В репо лежат: `.env`, `db.sqlite3`, папка `Postgres_db/` (данные PG), `node_modules/`, `__pycache__`, `src.zip`, `reviews.zip`, дубликат `frontend/src/src`.

Фикс: актуализировать `.gitignore`, убрать артефакты, секреты хранить только в `env`.

● Производительность / SQL

6. Ценовую фильтрацию мимо денормализованных полей

Файл: `apps/catalog/filters.py`

```
min_price = NumberFilter(field_name='variants__price__price', lookup_expr='gte')
```

JOIN на варианты + `DISTINCT`, без использования денормализованных `min_price/max_price` на `Product`, которые уже обновляются сигналом.

Фикс: фильтровать по `min_price__gte` / `max_price__lte` на самом `Product` — короткий SQL, индекс, без JOIN.

8. Поиск идёт по трём `__icontains`, а не по FTS

Файл: `apps/catalog/filters.py` (`filter_search`)

```
Q(name__icontains=value) | Q(description__icontains=value) |  
Q(brand__name__icontains=value)
```

`search_vector` (`SearchVectorField` + `GinIndex`) реализован и обновляется сигналом, но в фильтре не задействован.

Фикс:

- На PostgreSQL — `search_vector` (ранжирование с весами уже настроено).
 - На SQLite — fallback на `__icontains`.
-

9. Listing каталога без кэша

`GET /api/v1/catalog/products/` — публичный, с пагинацией и фильтрами.

- `PageNumberPagination` делает `COUNT(*) + SELECT ... LIMIT/OFFSET` на каждую страницу.
- Нет `CACHES` в settings, нет `cache_page` / `django.core.cache`.

Фикс: настроить Redis (бэкенд уже в requirements), кэшировать listing но короткий TTL 30–60 секунд с ключом по параметрам; на detail кэшировать по slug/UUID. Инкремент `views_count` через `F()` уже безопасен.

10. `generate_unique_slug` — цикл `exists()` на каждую итерацию

Файл: `apps/catalog/services/slug_service.py`

При занятом slug генерирует `-2`, `-3`, ... вызывая `exists()` на каждую итерацию ($O(N)$).

Фикс: при конфликте — короткий случайный/хэш-суффикс или `slugify` + `pk`; уникальность закрепить на уровне БД.

● Логика / целостность

11. Сумма оплаты не сверяется с заказом

Файл: `apps/payments/services/payment_service.py` (`create_payment`)

`amount` принимается «на веру», не сверяется с `order.total`.

Фикс: валидировать `amount == order.total` (серверная сумма, независимая от клиента).

12. `confirm_payment` глотает `except Exception`

```
try:
    OrderService.confirm(payment.order)
except Exception as exc:
    ...
```

Ловится всё подряд, включая `ValidationError` (например, не хватает стока). Заказ «зависает» в `PENDING` при полученных деньгах.

Фикс:

- Ловить только ожидаемые исключения (`ValidationError`).
 - Резерв склада — при оформлении заказа, а не после оплаты.
 - При недоступном стоке — отдельный flow возврата средств.
-

13. `cart.tasks` почти «заглушки»

Файл: `apps/cart/tasks.py`

`send_abandoned_cart_reminders` считает корзины, но ничего не отправляет (возвращает строку). Либо доделать реальную email-рассылку, либо убрать.

14. `OrderService.cancel` не пересчитывает скидку

Файл: `apps/orders/services/order_service.py` (`cancel`)

В докстрингах заявлен пересчёт `discount`, но метод лишь меняет статус и причину. Проверить возможность расхождения `total` при возврате скидки.

⚠️ Дополнительные замечания

- В `settings.py` закомментирован pg-пул (`OPTIONS.pool`) — для прода раскомментировать.
- Нет конфигурации `LOGGING` (структурированные `logger.info(...)` есть в сервисах, но не пишутся ни в файл, ни в Syslog).

- Сигналы (`search_vector`, пересчёт `min/max price`, пересчёт `rating`) выполняются синхронно и могут тормозить `save()` на больших каталогах — вынести в Celery-задачу.

Предложение по приоритету

#	Задача	Приоритет
1	<code>select_for_update</code> + <code>get_or_create</code>	●
2	Webhook-безопасность	●
3	JWT blacklist	●
4	Генерация <code>order_number</code>	●
7	Ценовый фильтр (денормализация)	●
8	Поиск через FTS	●
9	Кэш каталога	●
11	Сумма оплаты vs заказ	●
12	<code>confirm_payment</code> — обработка ошибок	●
14	<code>OrderService.cancel</code> — проверка скидки	●

Как дальше

1. По бодтверждении — взять группу критичных задач (●) и подготовить детальный план с точками изменения и тестами.
2. Затем — производительность (●) с замером SQL-запросов (`db.reset_queries`).
3. Логика (●) — после стабилизации безопасности и производительности.

Статус файла: создан в рамках аудита; содержимое отражает анализ на 21 августа 2026.